Behavior Programs as Measures over Proofs

Staged stochastic multiset rewriting with a linear-logic front end, an $\mathrm{MLL}^{\mathrm{B}}$ proof-net intermediate representation, and an executable, certified, machine-checked compilation pipeline

The metis project*

Draft 0.4 — 2026-10-27

Status markers. [proved] = proof in this paper; [mechanized] = machine-checked in Lean 4 in the public repository (directory `formal/`, https://github.com/metis-lang-dev/metis-lang); [pinned] = mechanically test-pinned in the repository; [sketch] = proof sketch, mechanization open; [open] = open problem.

Changes from 0.3. A new section on belief (§8): epistemic probability enters a program only through a finite port, a belief is a mixture over initial multisets (Theorem 9), the program reads it as a level atom, and its certificate gains a finite sheet and loses nothing (Proposition 10, Corollary 11). Interfaces, packages and compositions are now part of the language (§2.4), with the composite measure a product that the tick protocol determines (Proposition 5). The evaluation bound separates grounding, which is a join, from inference, which is the proof-net's (§9.2). The text has been rewritten for a reader seeking understanding: the history of the drafts and the engineering particulars are in the repository's record, not here.

Abstract

We present a behavior-planning framework in which a *program* — a catalog of typed, staged, weighted multiset-rewriting rules in the Ceptre lineage of intuitionistic linear logic — denotes a *probability measure over its own derivations*. Three readouts of one object coexist: proof theory (which derivations exist), operational semantics (how they interleave), and measure (how likely each is). We prove that the interpreter's run distribution and the denoted k-step dynamic Bayesian network agree exactly (run-level preservation), characterize the image of the denotation as stage-gated mass-action stochastic Petri nets (equivalence onto the image, now machine-checked in Lean 4), and show that for a statically-recognizable *factorizable class* the measure descends to the Mazurkiewicz trace quotient — runs weigh their occurrence nets, not their interleavings — connecting our compiled representation to the Bayesian proof-nets of Ehrhard, Faggian and Di Guardia read in the opposite direction: they evaluate one net where we select among many. Uncertainty about the world — epistemic, as opposed to the aleatoric weights inside a program — enters only through a finite port: a belief is a mixture over initial multisets, the program reads it as a level atom, and its certificate gains a finite sheet and loses nothing. The correspondence is not only semantic but *compiled*: programs lower through a typed ground normal form to a proof-net-shaped intermediate representation (CHOICE sites as CPT boxes, SSA atom occurrences as wires) on which typing — including machine-checked well-formedness as EFD-style Bayesian proof-nets — cost certificates and canonical hashing are IR-level analyses, and from which exact filtering, junction-tree elimination, fixed-budget sampling, tree search, a dependency-free C++ interpreter, and standard factor-graph export are generated back-ends. Admission-time cost certificates make the pipeline deployable in real-time physical systems: programs

*Sources, corpus, goldens, and the Lean mechanization: https://github.com/metis-lang-dev/metis-lang. Correspondence: info@robofortis.com.

are rejected or downgraded before evaluation, never killed mid-flight. We report exact parity chains (interpreter $\doteq$ filter $\doteq$ factored $\doteq$ exported artifact $\doteq$ C++ interpreter, at 10^{-9} across 200+ gates), provable optimality on IPPC benchmark instances, a growing Lean 4 mechanization of the metatheory, and honest limits (the 2^n support cliff and what the factored evaluator does and does not buy back).

1 Introduction

Two research programs meet in the middle of this paper.

Ehrhard, Faggian and Di Guardia (EFD) read a correspondence between Bayesian networks and multiplicative linear logic **right to left**: a BN's joint distribution *is* an $\mathrm{MLL}^{\mathrm{B}}$ proof-net (MLL plus probabilistic box axioms carrying CPTs), and exact inference *is* the inductive evaluation of that net in probabilistic coherence spaces, cost-matched to factor elimination [EFD24, EFD26]. One net, given by the model; the computational question is evaluation.

We read the same correspondence **left to right**. A linear-logic program does not denote one proof; it *generates a space of derivations* — plans, explanations, futures. Equipping the CHOICE points of the operational semantics with weights makes that space a probability space, and the interesting computations are over the ensemble: *sum* over derivations is filtering and prediction; *argmax* is most-probable explanation and planning; *reweighting by evidence* is diagnosis and behavior identification. Probability selects among proofs; proof theory generates them.

Two kinds of uncertainty. The weights on CHOICE are the dice inside a behavior: given the world, this is how the program acts. Call that part *aleatoric*; it is fixed when the program is certified, and tuned, when it is tuned, by counts. A second kind of uncertainty is about the world itself — where the cup is, whether the beast is hostile — and it is *epistemic*: it belongs to the agent's map and changes with every observation. A program with fixed weights is a certified Markov decision process. With the map's belief entering through a port, it is a belief MDP — over a *finite* quantisation, because the belief enters as a finite atom (§8). Keeping the two kinds apart, and letting the second cross into the first only through a finite port, is what lets every certificate of this paper survive the arrival of belief.

What the lollipop buys. For the experienced reader it pays to separate, at the door, the two halves of the logic–probability correspondence, because they are statements of different kinds. The *Boolean* half — events form an algebra, probability is a measure on it, $\wedge/\vee/\neg$ are $\cap/\cup/\complement$ — is classical [Kol33], geometrically exact, and structurally inert: it says which events exist and yields a flat joint measure, but nothing about how the joint *factors*. Everything substantive in this paper lives in the other half: the linear implication $A \multimap B$ read as the conditional $P(B \mid A)$. This is not another Venn-style equivalence; it is a statement about *direction and factorization*, which the event algebra cannot express at all. A BN node with its parents is one lollipop — EFD's box axiom: negative premises, one positive conclusion, a CPT, with $A \multimap B = A^{\perp} \parr B$ [Gir87] the reason implication and box share a shape; the chain rule $P(x_1 \dots x_n) = \prod_i P(x_i \mid \mathrm{pa}_i)$ is a composition of lollipops (in Markov-categorical terms: conditionals are directed morphisms and composition is the chain rule [CJ19, Fri20]); a DBN's transition kernel is one lollipop per slice. The reading is load-bearing three times over in what follows. The emission compiles every clause $c_r \multimap o_r$ into a CPT row whose parents are exactly the clause's premises, and the box-rule shape is machine-checked on every artifact (Proposition 7). The run measure factorizes per site into conditionals given exactly those premises (Theorem 4) — and that factorization, not the flat measure, is what the cost

certificates, junction trees and exports operate on. And the implication is *consuming*: co-enabled clauses compete for the same tokens, so conditioning on an outcome anticorrelates the alternatives — explaining-away *is* resource competition, priced by the falling factorials of Theorem 1 — a phenomenon the Boolean half is structurally blind to, since classical contraction lets a premise be used twice. The direction of $\multimap$ is likewise why conditioning remains a semantic operation on the measure rather than a syntactic one on proofs (§7, O3).

The two directions meet at the occurrence net: each of our runs, quotiented by interleaving, is an EFD-style net; our k-step measure is a weighted superposition of such nets; conditioning and MAP pick nets out of the superposition — a question that cannot arise on the EFD side, where a query has exactly one net.

The mission, stated plainly. The project's goal is a *formally provable* program — in both senses — from linear logic to probability graphs: a language whose denotation into factored probabilistic models is established by theorems, whose theorems are progressively machine-checked in a proof assistant, and whose compiled artifacts are certified by admission-time checkers so that what the deployed interpreter evaluates is exactly what the theorems govern. The theorems are on paper, the artifact checks run at every build, and the metatheory is progressively discharged in Lean 4 (§11). The three levels of assurance — paper proof, mechanized proof, mechanically pinned artifact — are marked per-statement throughout.

This paper contributes, on the theory side:

1. **Run-level preservation** (Theorem 1, [proved], counting kernel [mechanized]): the interpreter's run distribution and the denoted k-step DBN are related by a measure-preserving bijection; the falling-factorial CHOICE weights are exactly the token-individuation counts of the ordered-resource interpreter.

2. **Exactness of reachable-support filtering** (Theorem 2, [proved], [mechanized]): forward filtering over the reachable support is exact inference on the denoted DBN, not an approximation.

3. **Equivalence onto the image** (Theorem 3, [proved], [mechanized]): the denotation is an equivalence between ground staged programs modulo structural congruence and finite stage-gated mass-action stochastic Petri nets — the honest "isomorphism theorem"; logic $\leftrightarrow$ all-of-PGM is obstructed and we say exactly why (§7).

4. **Trace invariance on the factorizable class** (Theorem 4, [proved], descent skeleton [mechanized]): for programs admitted by our factored emission, the run weight factorizes per CHOICE site and is invariant under Mazurkiewicz trace equivalence — the measure is a measure on occurrence nets. Outside the class, per-slice normalizers are interleaving-dependent and the quotient fails in general; the class boundary is *statically recognizable and enforced*.

5. **Quiescence is compiled away** (Proposition 6, [proved]): the stage/quiescence discipline — the one construct of our source language outside MLL — is eliminated by the finite-horizon normal form for admitted programs: the IR is quiescence-free. The non-MLL residue is a *source-language* convenience, not a semantic commitment. Moreover the emitted IR is a well-typed **Bayesian proof-net** in the EFD sense, machine-checked at every serialization (Proposition 7, [proved], [pinned]) — the static well-formedness half of the Di Guardia typing question, whose compositional half remains Conjecture 8.

6. **Belief through a finite port** (Theorem 9, [proved], [mechanized]; Proposition 10, Corollary 11, [proved]): a categorical belief bound to a port makes the one-tick number

a mixture over initial multisets; with the belief entering through the presence draw alone the number is multilinear, so a finite cell sheet bounds it; the program's certificate is unchanged.

7. **The composite measure is a product** (Proposition 5, [proved]): under the tick protocol a composition of certified packages has the product of their measures, so nothing composite need be grounded to certify it.

And on the systems side:

8. A **compilation pipeline** (§9) with the shape of knowledge compilation: `.llp` source → typed ground/unrolled normal form → MLL^{B}-shaped IR (sites + SSA occurrences) → generated back-ends. Slogan: MLL^{B} *is to linear-logic multiset rewriting what SDD circuits are to probabilistic logic programming.*

9. **Admission-time cost certificates** (§9.4): induced width and total factor-ops computed symbolically from IR scopes before any table exists — the real-time defense doctrine (reject or downgrade the evaluator ladder at admission; never kill mid-flight).

10. A **verification-translator architecture** (§9.6): the compiler (`metisc`, OCaml) and the Python referee toolkit are certifier and cross-check; the deployed C++ runtime is a ∼300-line dependency-free interpreter of the compiled artifact, pinned by golden parity and by external referees.

11. A **mechanization** (§11): a Lean 4 development, shipped in the public language repository, machine-checking Theorem 3 in full and the cores of Theorems 1, 2 and 4. The paper contains no Lean code; statements cited as [mechanized] name their Lean identifiers in §11.

What is *not* claimed: novelty of the static BN↔proof-net correspondence (that is EFD's, published first — see §12 for the audit trail); an isomorphism between the logic and all of PGM (obstructed, §7); completeness of the learning loop (doctrine and seams exist; two helpers remain unbuilt, §10).

2 The source language and its operational semantics

2.1 Programs

A **catalog** declares finite types, namespaces with layer rights, predicate signatures, backward (Horn) predicates, ports, and entries. Entries are clause *schemas* and link *schemas* over typed variables, each carrying a mandatory human-approvable comment and a provenance record (`hand` | `surprise` | `template`). **Grounding** Π-instantiates schemas over the signature, prunes by Horn-derivable guards and distinctness constraints, and yields a **ground program**

$$P = (S,\ (C_s)_{s\in S},\ L,\ s_0,\ m_0)$$

with stages S (ordered), ground clauses C_s per stage, ground links L, initial stage s_0 and initial multiset m_0. A ground clause

$$r: \quad c_r \multimap o_r \quad [\text{persist } p_r] \quad \text{weight } w_r$$

consumes multiset c_r, reads (bang-style, never decremented) p_r, produces o_r, and carries a prior weight $w_r \in \mathbb{R}_+$, optionally bound through a **weight port**: a named factor whose implementation lives in a registry, with a *declared read-scope* `reads {…}` (§9.3). A link is a clause that additionally switches the active stage and is gated on quiescence.

2.2 States and the three-tier CHOICE

A state is $\sigma = (s, m)$: active stage and multiset. One step:

- **Tier 0 (clauses).** Enabled clauses of s draw

$$W_r(m) = w_r \cdot \varphi(m, r), \qquad \varphi(m, r) = \prod_a m(a)^{\underline{c_r(a)}} \cdot \prod_a m(a)^{p_r(a)}$$

  the falling-factorial transition count (consume) times the power count (reads). One enabled clause is chosen with probability $W_r / \sum_{r'} W_{r'}$. A **policy hook** may reweight or prune this tier (safety shields, behavior rows); the measure renormalizes.

- **Tier 1 (links).** If no clause of s is enabled (*quiescence*), enabled links of s compete the same way and switch the stage.

- **Tier 2 (done).** Otherwise the state is absorbing.

UPDATE is deterministic given the chosen event. A **run** of horizon K is the sequence of K chosen events (with `done` self-loops); the **run measure** μ_K is the product of its steps' CHOICE probabilities.

This is the Ceptre operational core [Mar15] — itself the first-order Horn fragment of intuitionistic linear logic as multiset rewriting [CS09] — extended with (i) the weight priors as first-class syntax, (ii) weight ports with declared scopes, and (iii) the stage/quiescence discipline as tiered CHOICE.

2.3 Three readouts, one object

The same P supports: **proof theory** — runs are derivations; each run's dependency structure is an occurrence net (§6); **operations** — the interpreter samples runs; **measure** — μ_K, with filtering, smoothing, MAP, hypothesis posteriors and surprise as queries. Nothing is layered on top of anything: all three are readouts of P.

2.4 Interfaces, packages, compositions

How is a large behavior written? Not as one program. A robot's navigation, grasping and task logic are separate *specialists*, each a program of its own, and a *director* that hands goals down and reads reports up. The language keeps them separate and fixes, in declarations, exactly what crosses between them.

The forward declaration is small. An **interface package** is a lifted file with only declarations: abstract types and predicates over them, no rules, no facts, no stages, with its own name and canonical key. Every predicate two programs share is declared *once*, there, and carries a *nature*: a *signal* (a world reading or a crossing vocabulary — *goto*, *reached*), a *resource* (the hands, the position: held by the director, granted as a lease), or a *report* (status, done, stuck). A specialist includes the interface packages it speaks and states its *direction* on each predicate — in, out, resource, or world-private — and a static check refuses a direction the nature does not allow, a shared predicate declared locally, or two interfaces declaring one name.

The package is the certified unit. A **package** is a program with its manifest, its cases (an initial multiset, a script of deliveries, the expected atoms) and its goldens. Its certificate is the program's canonical key plus its cases passing against one *generic* interface stub that knows, from the manifest alone, what to feed (the in-set), what to collect (the out-set), what to grant and how to score. A specialist's certificate therefore never depends on the director it will meet: the protocol it speaks is a declaration package with no rules.

The shared atoms are the runtime's. A **composition** is a set of packages over interface packages under a *tick protocol*: inputs are taken at a tick boundary, outputs are latched to the next boundary, the relay between members is deterministic, and each member samples its own stream. Shared state crosses as a *copy per port* at the boundary, never as a live shared truth: a linear-logic $!p$ readable by every rule and consumed by none would make a trigger that never latches and a slow member that sees a moving target. The composition's certificate is its package key, the verdict of the interface checks, and each member's own key in manifest order — nothing composite is grounded, because the composite measure is a product the protocol determines (Proposition 5).

A **bundle** is a composition sealed for one device with a runtime manifest: which runtime executes each member, where it is placed, and which host library binds each of its ports, each library with its own key. A library upgrade changes the bundle's runtime key and not the composition's; a member may be replaced over the air iff it was certified against the same interface keys and the composition passes its checks again, which is a static and cheap test.

3 Run-level preservation

The denoted **k-step DBN** has variables $X@t$ (the event chosen at step t, including `quiesce`/`done`) and $K@t$ (the multiset state), with conditionals *defined* by §2.2's CHOICE and deterministic UPDATE. The substantive theorem relates this to the *finer-grained* semantics an ordered-resource interpreter (Ceptre's) actually executes, where a step chooses not a ground clause but an **instance**: a clause together with an injective assignment of concrete resource tokens to its consume slots and a choice of token per read occurrence.

Theorem 1 (preservation; [proved], counting kernel [mechanized])**.** *Fix a horizon K. The map that sends an interpreter trajectory (sequence of instances) to its event sequence (sequence of ground clauses) pushes the interpreter's trajectory distribution forward to μ_K, and the induced map on state trajectories is a measure-preserving bijection between the interpreter process and the DBN process.*

Proof. Both processes share state space and deterministic UPDATE, so it suffices to show per-step agreement of the choice distributions; the trajectory statement follows by induction on K with the product decomposition of both measures.

Fix $\sigma = (s, m)$ at tier 0 (tiers 1–2 are identical on both sides, as links carry the same count structure and `done` is deterministic). On the interpreter side, tokens are individuated: the multiset m is a set of tokens with $m(a)$ tokens of species a. An instance of clause r chooses, for each species a, an *injective* assignment of $c_r(a)$ consume slots to distinct tokens of a — there are $m(a)\,(m(a)-1)\cdots(m(a)-c_r(a)+1) = m(a)^{\underline{c_r(a)}}$ such assignments — and, independently, one token per read occurrence — $m(a)^{p_r(a)}$ choices. (Equivalently: an *unordered* choice of $c_r(a)$ distinct tokens followed by an ordering onto the slots, $\binom{m(a)}{c_r(a)} \cdot c_r(a)! = m(a)^{\underline{c_r(a)}}$ — the count is the same whether one individuates slots or only tokens, which is why the falling factorial is forced and not a convention.) The instance count of r at m is therefore exactly $\varphi(m, r)$. The interpreter draws instances with probability proportional to the clause

prior w_r per instance; hence the total probability of *some* instance of r is proportional to $w_r \cdot \varphi(m, r) = W_r(m)$, which is the DBN's CHOICE weight. All instances of a fixed r produce the same successor multiset (removing c_r, adding o_r is species-level), so the pushforward along instance → clause is exact, and distinct clauses map to their own W_r mass. Normalizing over the enabled set gives equality of the one-step kernels. □

Two corollaries worth naming. First, *substitution collapse pricing*: when grounding identifies two variables ($X{=}Y$), the ground clause's consume multiset gains multiplicity 2 and φ prices it as a falling factorial — the theorem shows this is forced by token individuation, not a modeling choice. Second, *explaining away = resource competition*: the collider at $X@t$ (parents = the consumed species) anti-correlates alternative consumers of the same tokens under conditioning, priced exactly by the same counts. [pinned]: oracle parity $\leq 10^{-12}$ on the analytic fixtures, $\leq 10^{-9}$ on the road corpus, across all horizons. The counting kernel — instances of r at m are equinumerous with $\varphi(m, r)$, and the instance-to-clause pushforward carries weight $w_r\,\varphi(m, r)$ — is [mechanized] (§11).

Theorem 2 (exact filtering; [proved], [mechanized])**.** *Let D_t be the forward filter's distribution over states after t steps, computed over the reachable support only. Then D_t equals the DBN's exact marginal at slice t.*

Proof. Induction on t. D_0 is the point mass at (s_0, m_0). The DBN's slice-$(t{+}1)$ marginal is $\sum_\sigma P(\sigma' \mid \sigma)\, D_t(\sigma)$ by the law of total probability; because UPDATE is deterministic, $P(\sigma' \mid \sigma)$ is supported on finitely many σ' per σ, and every σ with $D_t(\sigma) > 0$ is by construction in the filter's support. The filter computes exactly this sum. No mass ever leaves the reachable set, so restriction loses nothing. □

The point of stating the triviality: the filter is *not* an approximation of PGM inference — it is variable elimination in the one elimination order (time-forward) whose intermediate factors are supported on reachable states. Its failure mode is honest: support grows like 2^n on synchronous-parallel domains (the IPPC cliff, §9.5), which is what the factored back-end addresses.

4 Canonical identity: programs and runs

Two identities, two quotients [pinned]:

Programs mod presentation. `canonical()` content-sorts clauses, links and names; `program_key()` hashes the result (stage order and initial multiset retained — they are semantics). Invariant under presentation shuffles; discriminates weight, multiplicity, stage wiring, init and dropped-clause mutations. This key is the cache key for every compiled artifact (§9.4) — canonical identity is not bookkeeping, it is what makes AOT compilation sound.

Runs mod interleaving. Independence of events is footprint disjointness — writes (consume ∪ produce) conflict with everything, reads conflict only with writes — with the stage discipline encoded as an implicit resource (clauses read `stage@s`; links write both endpoint stage tokens), so cross-stage ordering falls out of the same rule. `run_key()` hashes the **Foata normal form** of the trace: the canonical layering of the run's occurrence net. Two interleavings of the same concurrent run share one key; symmetric ground instances collapse — a quotient we want.

The delicate question — *does the measure respect this quotient?* — is §6, and the answer is one of the paper's main theorems.

5 Equivalence onto the image

No isomorphism between the logic and all of PGM exists at the implementable level; §7 gives the three obstructions. The honest strong statement characterizes the image.

Definition (the image class)**.** A **stage-gated mass-action SPN** is a tuple $N = (A, \bar{S}, T, G, s_0, m_0)$ where A is a finite set of places (ground atoms); $\bar{S}$ is a finite *ordered list* of stage labels (stages with no transitions are retained — they are declared structure, not derived); T is a finite **multiset** of *stage transitions*, each a quadruple (s, c, p, o, w) of a stage label, consume/read/produce arc multiplicity vectors over A, and a rate constant $w \in \mathbb{R}_+$ (a multiset, because nothing forbids two indistinguishable transitions, and the firing rule counts them separately); G is a finite multiset of *gate transitions* (s, s', c, p, o, w) additionally carrying a target stage; $s_0 \in \bar{S}$ and m_0 is a marking. The propensity of a transition at marking m is w times the mass-action count (falling factorials on consumed places, powers on read places), and the firing rule is §2.2 verbatim: stage transitions of the active stage compete first, gates fire at stage quiescence and set the register, normalization per step.

Definition (structural congruence)**.** $P \equiv P'$ iff P' is obtained from P by bijectively renaming clause and link *names* and reordering the clause and link lists. Atoms, stage labels, stage order, weights, and the initial data are fixed pointwise — they are semantics, not presentation (renaming an atom or a stage changes which SPN is denoted; renaming a clause does not).

Theorem 3 (equivalence onto the image; [proved], [mechanized])**.** *The denotation $[\![\cdot]\!]$ from ground staged programs to stage-gated mass-action SPNs is (i) total and functional, and constant on congruence classes; (ii) surjective; (iii) injective up to structural congruence. Hence it descends to a bijection between programs-mod-congruence and the class.*

Proof. (i) *Totality and invariance.* $[\![P]\!]$ is the §2 construction: places = the (finitely many) ground atoms of the signature, $\bar{S}$ = the declared stage list in declared order, each ground clause r of stage s contributes the stage transition (s, c_r, p_r, o_r, w_r), each link the corresponding gate, s_0 and m_0 carried over. Every component of the tuple is assembled from the *content* of clauses and links — names appear nowhere, and T, G are multisets, so list order appears nowhere either. Hence $[\![\cdot]\!]$ is total, functional, and constant on congruence classes.

(ii) *Surjectivity.* Given N in the class, define a reading-back $U(N)$: fix any enumeration $t_1, \ldots, t_k$ of the multiset T (duplicates enumerated with multiplicity) and any enumeration $g_1, \ldots, g_l$ of G; let $U(N)$ have stages $\bar{S}$ in order, clause r_i named freshly (say c_i) with stage, multisets and weight copied from t_i, link l_j named freshly with endpoints and data copied from g_j, and initial data (s_0, m_0). Then $[\![U(N)]\!] = N$ *on the nose*: each tuple component round-trips by construction, and the operational behaviors agree because the class's firing rule was *defined* to be §2.2. (The choice of enumeration changes $U(N)$ only up to $\equiv$, which (i) already quotients away.)

(iii) *Injectivity up to congruence.* Suppose $[\![P]\!] = [\![P']\!] = N$. From N we recover: the stage list with its order (so P and P' declare the same stages, including empty ones), s_0 and m_0, and — per stage label s — the multiset of content quadruples (c, p, o, w) of the clauses of s, and the multiset of gate contents (s, s', c, p, o, w) of the links. For each stage s, the clause lists of P and P' over s induce equal content multisets, so there is a bijection between them matching equal content; where several clauses share identical content, any matching of the duplicates will do, because identical-content clauses are fully interchangeable — swapping them is itself a congruence. The same argument gives a per-endpoint-pair matching of links: a link's endpoints (s, s') are part of its *content*, so the matching automatically respects the stage graph — this is the point where renaming might have interacted with the stage-linking structure, and it cannot: clause and link names occur nowhere else in a program (clauses do

not reference each other or links by name; links reference *stages*, and stages are not renamed), so the per-stage and per-link matchings assemble into one global renaming-plus-reordering with no cross-boundary constraint left to check. That renaming exhibits $P \equiv P'$. □

Two boundary remarks the proof depends on. *Empty stages*: $\bar{S}$ retains them; had the SPN carried stages only as the partition induced by T, a declared-but-unpopulated stage would be forgotten and (iii) would fail (two programs differing by an unreachable empty stage would collide) — the class definition is chosen so that the denotation forgets *exactly* presentation. *Duplicate transitions*: T is a multiset for the same reason — two identical-content clauses double the propensity of that content, which a set-valued T would silently halve. The theorem is [mechanized] (§11): programs as name-carrying lists, the class as multiset-valued tuples, congruence as renaming plus permutation, with totality, invariance, surjectivity (an explicit fresh-naming section of $[\![\cdot]\!]$) and injectivity-up-to-congruence each a named Lean result. The [pinned] evidence is a fuzz test of the same statement on the implementation: the canonical key is invariant under presentation shuffles and discriminates every semantic mutation.

Classical context: the $\otimes$-fragment $\leftrightarrow$ Petri reachability correspondence is Gunter–Gehlot [GG89] and Engberg–Winskel [EW93]; MSR as the Horn fragment of ILL is Cervesato–Scedrov [CS09]; the mass-action propensity shape is Gillespie's [Gil77]. The assembly — *staged*, *gated*, *discrete-time normalized*, with an executable preservation test and a mechanized equivalence — is ours.

6 The measure on occurrence nets: trace invariance

Here is the problem the seed analysis left open. The engine weights $W_r(m)$ are local, but each step's *normalizer* $\sum W_{r'}$ ranges over everything enabled at that step — and what is co-enabled depends on the interleaving. In general, two interleavings of the same occurrence net can carry different products of normalized probabilities: the measure lives on sequences, not nets. (Abbes–Benveniste's branching cells [AB06] are the general repair; importing them wholesale is [open].)

The factored-emission work yields a large, statically-recognizable class where the repair is free.

Definition (factorizable class $\mathcal{F}$)**.** A ground program (with its horizon and declared port scopes) is in $\mathcal{F}$ iff the factored emission (§9.3) admits it: (a) all multiset counts are 0/1; (b) the stage/tier schedule is branch-independent (which micro-step runs which competition is the same on every run); (c) at each micro-step, the candidate set forms a **site**: every possibly-enabled event consumes the same anchor token, and each candidate's enabledness and weight depend only on a *scope* — its consumed/read atoms plus its port's declared `reads` scope; (d) no event produces an atom that may already be present (checked statically or by the runtime mass check).

Theorem 4 (trace invariance on $\mathcal{F}$; [proved], descent skeleton [mechanized])**.** *For $P \in \mathcal{F}$, the weight of a run factorizes as*

$$\mu(run) \;=\; \prod_{\text{sites } \sigma \text{ visited}} P(chosen_\sigma \mid scope\text{-}valuation_\sigma)$$

and is invariant under Mazurkiewicz trace equivalence. Consequently μ_K descends to a measure on occurrence nets: `run_key` classes are measurable atoms, and the k-step measure is a weighted sum over EFD-style nets.

Proof. By (b), every run visits the same sequence of sites, so the product ranges over a common index set; it remains to show each factor is well-defined on the trace class. Fix a site σ and consider two runs in one trace class. Their prefixes before σ differ by a sequence of adjacent transpositions of independent events (Mazurkiewicz).

Lemma (footprint stability). If events e, e' are independent (disjoint footprints in the §4 sense, including stage tokens), then executing e' does not change (i) the truth of any atom in e's site scope, nor (ii) e's enabledness. — Immediate: e's scope atoms are among its consumed/read/declared atoms; independence says e' writes none of them.

By the lemma and induction on transpositions, the scope valuation at site σ is equal in both runs. The site's normalizer is $\sum_{\text{candidates}}(\text{enabledness} \times w \times \text{port-factor})$, and by (c) every term is a function of that scope valuation alone — *this is exactly what the declared* `reads` *contract buys*: an opaque binding reading undeclared global state would break the lemma, which is why the emission rejects undeclared ports. Hence each factor agrees; the products agree. With (a), $\varphi \equiv 1$, so no falling-factorial term reintroduces marking dependence. □

Three remarks. First, the theorem *identifies our sites with branching cells* on $\mathcal{F}$: local competition arenas whose choices are independent of scheduling — "concurrency = probabilistic independence" made constructive by a compiler check. Second, the class boundary is *enforced, not assumed*: static admission plus the runtime total-mass check ($Z = 1$) convert any reachable violation into an error rather than a silent approximation ([pinned]: rejection gates). Third, outside $\mathcal{F}$ the engine semantics remains well-defined (Theorems 1–2); what fails is only the descent to nets — normalizers become schedule-dependent, which is precisely the interleaving junk the general theory must quotient by force. The descent skeleton — run weights that factor through schedule-stable local factors are invariant under adjacent transposition of independent events, hence constant on Mazurkiewicz classes — is [mechanized] in abstract form (§11); the site/scope instantiation for the concrete emission remains on paper.

The composite measure factorizes by the tick. The same descent holds one level up, for a composition of packages (§2.4).

Proposition 5 (the composite measure is a product; [proved])**.** *Let a composition run under the tick protocol: (F1) within tick t a beating member d reads only its private state and the copies taken for it at boundary t; (F2) nothing d emits at t is visible to another member before boundary $t{+}1$; (F3) the copy, relay and grant transfer at a boundary is a deterministic function of the members' latched outputs and the world's readings; (F4) the members' random streams are disjoint. Then for every composite trace τ,*

$$P(\tau) \;=\; \prod_{t} \; \prod_{d \text{ beats at } t} P_d\big(\text{steps of } d \text{ at } t \;\big|\; \text{state}_d(t)\big),$$

where $\text{state}_d(t)$ is d's private state with its boundary-t copies, fixed by τ's prefix through (F3), and each P_d is the member's own run measure μ_K of §2.2.

Proof. By (F1) and (F2), no member's CHOICE within a tick depends on another member's CHOICE in the same tick; by (F4) the draws are independent; by (F3) the state each member starts a tick from is a function of the prefix. The probability of the tick's joint step is therefore the product over beating members of their own one-tick measures, and the product over ticks follows by the chain rule. □

What the product does *not* give is the flat program's measure: interleavings within a tick are not represented — (F2) forbids them — and a chain of three members takes two boundaries and shows as such. That is the price of certifying members alone, and it is paid in latency, not in semantics. Arbitration is the director's CHOICE at its own beat, inside its own measure; the grant it emits reaches the holder at the next boundary.

7 Why no logic ↔ PGM isomorphism, and what quiescence really is

O1 — interleaving junk. Sequence-level artifacts can only be homomorphic-with-preservation; the isomorphism-grade statement lives at the proof-net quotient — exactly where EFD obtain theirs, and where Theorem 4 puts us for $\mathcal{F}$.

O2 — copy/discard asymmetry. The PGM world is a Markov category [Fri20, CJ19]: copying and marginalization are free. Linear logic licenses contraction/weakening only under ! — our persistent read is the licensed copy; dropped products are weakening. A functor from a resource-sensitive syntax into a semicartesian target cannot be inverted on the whole target; the honest repair is Theorem 3's image characterization.

O3 — inference direction. $\multimap$ fixes the generative direction; a BN's defining extra power is conditioning (Bayesian inversion). Our system computes posteriors *about* derivations (smoothing, MAP, hypothesis posteriors, all exact over the reachable support [pinned]) but the lollipop has no proof-theoretic inverse here — inversion is a semantic operation on the measure, not a syntactic one on proofs.

Quiescence beyond MLL — and Proposition 6. The stage-token encoding (§4) shows stages alone are MLL-typeable sugar: clauses read `stage@s`, links consume/produce stage tokens. The residue is the quiescence gate — "fire only if *nothing* in this stage can" — a negative side-condition (inhibitor arc; negation-as-failure) invisible to any multiset of premises, hence outside MLL and outside the EFD setting. This was our honest deviation. The compiled view dissolves it:

Proposition 6 (quiescence is compiled away on $\mathcal{F}$; [proved])**.** *For $P \in \mathcal{F}$ at horizon K, tier decisions are branch-independent, so the normal form resolves every quiescence test statically; the emitted IR (sites + SSA occurrences + CPT boxes) contains no quiescence construct, and is* $\mathrm{MLL}^{\mathrm{B}}$*-shaped: boxes = sites (negative premises = scope occurrences, positive conclusion = outcome), contraction = SSA fan-out of an occurrence read by several later scopes, weakening = occurrences never read again.*

Proof. By $\mathcal{F}$(b) the schedule — including which micro-steps are link-tier — is a function of the step index alone, computed once by the admission analysis; the emission then never consults enabledness at evaluation time. [pinned] by the emission gates: mixed-tier programs are rejected, admitted programs carry no tier logic in the artifact. □

So "quiescence beyond MLL" resolves into: a *source-level* convenience (genuinely non-MLL, kept because staged authoring is how humans write behavior) that the compiler eliminates for the class it certifies. The general-class proof theory of quiescence (fixpoints? subexponentials? inhibitor semantics?) remains [open].

Proposition 7 (the emitted nets are Bayesian proof-nets; [proved], [pinned])**.** *For every admitted program and horizon, the emitted IR, read as an* $\mathrm{MLL}^{\mathrm{B}}$ *typed graph — CHOICE sites and SSA writes as probabilistic boxes (negative premises = the parent occurrences of the declared scope, positive main conclusion = the outcome/occurrence variable),* `unknowns` *lifts as parentless prior boxes, double-production guards as observed constraint boxes, atom fan-out as contraction on negative atoms, unread occurrences as the net's conclusions Δ — is a* ***Bayesian proof-net*** *in the sense of [EFD26, Defs. 3.1–3.2]: (a) every box satisfies the box rule (all conclusions atomic, exactly one positive, $X \notin \{Y_i\}$); (b) box conclusions are pairwise distinct; (c) every premise edge is the conclusion of some box (no pending premises); (d) the net is correct — by [EFD26, Lemma 3.1] it suffices that the polarized orientation is a DAG; (e) all conclusions in Δ are positive. Site CPT rows are normalized or zero (stall rows,*

convertible to errors only by the runtime $Z = 1$ check, §9.3); write boxes are deterministic one-hot; guards are 0/1 indicators.

Proof. Each condition holds by construction of the emission (§9.3): (b) is SSA single-writer — every occurrence variable is minted exactly once, every site variable is fresh per micro-step; (c) holds because a variable enters a scope only through the environment map, whose non-constant entries are exactly the minted variables; (a) holds because a site never reads its own outcome and a write never reads the occurrence it mints; (d) holds because every premise of a box minted at micro-step t was minted strictly earlier, so emission order is a topological order of the polarized orientation; (e) holds because pending edges are unread box main conclusions, which are positive by the box rule. □

The point of *also* pinning it: the construction argument is about the emission as designed; the checker verifies conditions (a)–(e) mechanically on the emitted artifact itself — admission-time, symbolic, zero binding calls, same doctrine as the cost certificate — and runs both over the IR golden corpus (plus the unknowns and guard constructions) and *at every IR serialization*: the emitter refuses to write an ill-typed artifact, so the deployed C++ interpreter — which by design never re-types (§9.6) — only ever receives certified nets. Adversarial mutations (a self-parent box, a double writer, a dangling premise, a polarized cycle, a sub-normalized CPT row) are each rejected, so the gate is not vacuous. Any future emission bug that breaks well-typedness fails CI and the wire, not a reviewer.

Conjecture 8 (Di Guardia compositional typing)**.** *Proposition 7 discharges the* static *half of the typing question: our emitted nets are well-formed Bayesian proof-nets, machine-checked. What remains is the* compositional *half — checking the emitted nets against the well-behaved-composition typing derivations of [DEF26] (their type system governs how nets compose via cut; our sites are their boxes, SSA fan-out their contraction, and Proposition 6 removes the structural moves the dynamic quiescence gate would have needed, so we conjecture every emitted net types). Mechanizing their typing rules over our artifact is the concrete next theory task. [open]*

8 Belief: epistemic probability through a finite port

What does a program do when the world is not fully known? Everything so far prices *what the program does*: the CHOICE weights are the aleatoric part, the dice inside the behavior. A second kind of uncertainty lives outside the program — *what the world is* — and it is epistemic: it changes as the agent observes. This section says how the second kind enters without touching the first, and why the certificates survive its arrival.

8.1 The example, and the two missing factors

A robot must fetch a cup. The cup is in one of three receptacles, and the agent's map gives a belief: 0.6 in the drawer, 0.3 on the shelf, 0.1 in the bin. The program has a rule that sends the robot to a receptacle,

$$go: \quad want(O) \otimes start \otimes !inrecep(O,R) \multimap want(O) \otimes goto(R) \qquad \texttt{@w look}(O,R),$$

whose weight is a **port**: a named factor the host binds, here to the belief. With the map asserting *inrecep* for every place the cup might be, the rule is enabled three times and the port prices the choice among them. This is the right program and the wrong probability. A CHOICE weight is normalized over the enabled events of a state (§2.2); it answers *where does the robot go first*, not *where is the cup*. Two things a total probability needs are absent:

- *The world factor.* "Where is the cup" is one distribution over three mutually exclusive facts. Three independent weights describe 2^3 worlds, including the cup in two places and in none.
- *The product.* $P(\text{fetched within } h) = \sum_R P(\text{cup in } R) \cdot P(\text{fetched within } h \mid \text{cup in } R)$. The first factor is the belief; the second is the program's own measure. Nothing in §2–6 multiplies them: a run starts from one initial multiset.

The rest of the section supplies the two factors (§8.2, §8.5), brings the product back into the program as a finite atom (§8.4), and proves that the program's certificate is unchanged by the arrival (§8.6).

8.2 The categorical port

A **belief group** is declared over a port that already exists, by naming which of the port's arguments is the one that varies: *for each object O, exactly one receptacle R holds, and* `look`$(O, \cdot)$ *is its distribution.* The group's key is the other arguments; its members are the values of the named one. Three forms cover what the maps produce: `one` (exactly one member holds; the masses sum to 1), `lone` (at most one; the residual is the world where none holds — the cup is lost), and an independent atom (present with its own probability). The host binds the port as before, one value per argument tuple; the only new obligation is the sum, and it is checked exactly, never silently renormalized. A distribution the host did not mean is refused, not repaired.

The declaration lives beside the program, not in it. A belief is a fact *about the program's inputs*; the tick — the one step the interpreter samples — never reads it, so the program and its canonical key (§4) do not move. What moves is the certificate's key, which takes the canonical line of the belief declarations (§8.6).

In proof-net terms (§7, Proposition 7) a group is *one* categorical box $\vdash X^+$ whose variable has the group's k members as values and the port as its CPT. Three Bernoulli boxes denote the same distribution only once an "exactly one" factor couples all three; the categorical box is one variable of cardinality k. The correct total and the cheaper net are the same object.

8.3 Belief facts and the world mixture

In the instrument of §8.5 the presence of a belief pred's atoms is not read from the initial multiset: it is *drawn* from the port. Each `one` group puts exactly one member present, with its probability; each `lone` group one or none; each independent atom is present or absent. A **world** w is one outcome of all the draws, with probability $P(w)$ the product over groups, and the instrument starts from the mixture

$$\mu_0 \;=\; \sum_w P(w)\,\delta\big((s_0,\; m_0 \uplus w)\big).$$

Two disciplines keep this honest. A case's initial multiset may not list a belief atom (the draw puts it there). And belief preds are resource preds, never Horn guards: a guard is eliminated at grounding (§9.2), and a world must not change the ground program — every world shares one program and differs only in its initial multiset.

A read of a belief fact is conditioned on the truth. In the world where the cup is on the shelf, a rule enabled by $!\mathit{inrecep}(O, \mathit{shelf})$ fires there and nowhere else. That is right for the *world's* rules (the observation "absent", the find). It is omniscient for the *agent's* decisions: a task whose *go* reads the belief fact walks straight to the true place in every world, and its success is overstated. The honest program keeps two preds — the agent's *support* (where the cup may be, read by the decision) and the belief fact (where it is, read by the world) — and

one port prices both: as a CHOICE weight on the decision and as the presence of the truth. The instrument lists every rule that reads a belief pred, so the author sees which decisions are conditioned on the hidden truth.

8.4 Level ports: the only belief-derived input of the tick

The program never reads a number. A **level port** turns a number — a belief's value, or a query's answer — into a finite atom by declared thresholds: with cut points $0 < a_1 < \cdots < a_n < 1$ and names $\ell_0, \ldots, \ell_n$, the value p has level ℓ_i iff $a_i \le p < a_{i+1}$ (with $a_0 = 0$, $a_{n+1} = \infty$), compared exactly. The host asserts exactly one level atom per key and retracts the previous one; it is a world reading like any other. A tick program that reads only levels and static weights is a *finite* program, and finiteness is what §8.6 certifies.

This is the whole design in one sentence: the belief enters the program as a finite atom, so the program with belief is a program of the kind every theorem above already governs; the arithmetic that produced the atom lives in an instrument beside the program, where it can be exact.

8.5 The one-tick instrument

A **query** names an outcome and a horizon: the probability that Q holds at some step $t \le h$ (first passage; Q read as wanted or as a violation), or its presence at step h. The instrument answers for the beliefs *as they stand at this tick*:

- *Beliefs are constant over the horizon.* The port table is evaluated once, before step 0, and every weight port prices its events by that frozen table at every step. The map's own dynamics — the cup carried away, the plant drying — and its updates from what was seen stay in the map; the next tick's table comes from there.
- *Filtering stays in the map.* Within the horizon, what the program observes is conditioned exactly by the enumeration itself: in world w every observation follows w's truth.

The number is a sum-product over worlds and traces: products along a trace (the draws, then the normalized CHOICEs of §2.2), sums where states meet; a state where Q holds is absorbed, its mass added to the answer. It is the forward filter of Theorem 2 started from a mixture instead of a point.

The state budget is the cut. The filter retains at most N states, expanded in canonical order; the mass of what cannot be expanded is set aside, and the answer is an interval

$$lo = HIT, \qquad hi = HIT + CUT,$$

sound for first passage because the cut mass may or may not reach Q and nothing else can ([mechanized]: the dropped mass is moved by a stochastic kernel, never created or destroyed). A query is answered in bounded work at any horizon, and the width $hi - lo$ says how much the budget hid. With no cut, $lo = hi$ is the exact number — an exact rational: every weight, port value and threshold enters as the exact decimal of its shortest round-trip representation, so 0.7 is 7/10 and a distribution written 0.7, 0.3 sums to one, and the result does not depend on summation order. The brute-force referee that enumerates every path agrees to the last digit by construction.

8.6 What is proved

Theorem 9 (a port is a mixture; [proved], [mechanized])**.** *Let P be a ground program with belief groups B bound by f, and let μ_w be the first-passage measure of P started from*

$(s_0, m_0 \uplus w)$ *under the frozen table* f*. For every query* Q*, horizon* h *and budget* N*,*

$$lo \;\le\; \sum_w P_f(w)\,\mu_w(Q \text{ within } h) \;\le\; hi,$$

with equality on both sides — the instrument's number is the mixture — when no state is cut.

Proof. The instrument's recursion is Theorem 2's forward filter applied to the distribution $\mu_0 = \sum_w P_f(w)\,\delta(s_0, m_0 \uplus w)$ with Q-states absorbing. The one-step kernel is the same for every world (the worlds share the ground program and the frozen table), so the filter is linear in its initial distribution: $F^h(\sum_w P_f(w)\,\delta_w) = \sum_w P_f(w)\,F^h(\delta_w)$, and absorption commutes with the sum because it is a per-state operation. Each summand is $\mu_w(Q \text{ within } h)$ by Theorem 2. With a cut, the retained recursion is a sub-filter that drops nonnegative mass; the dropped mass is exactly CUT, and since the kernel is stochastic, what it would have contributed to HIT lies in $[0, CUT]$. □

The theorem says that a quantised port adds nothing the static model could not already express: it names *which* initial multisets a belief chooses and with what mass. Everything the earlier sections proved about one run from one initial multiset is inherited by the mixture, term by term.

Proposition 10 (multilinearity, and the cell table; [proved])**.** *Suppose the belief enters the number only through the presence draw (no* CHOICE *weight reads a belief port). Then* $\sum_w P_f(w)\,\mu_w(Q)$ *is multilinear in the groups' distributions: linear in each* $f(g,\cdot)$ *separately. Consequently, over any product of polytopes — one per group, cut out by the level thresholds of §8.4 — its minimum and maximum are attained at products of vertices.*

Proof. $P_f(w) = \prod_g f(g, w_g)$ is a product with one factor per group, and μ_w does not depend on f by hypothesis. A function of the form $\sum_w \prod_g f(g, w_g)\,c_w$ is linear in each factor's distribution when the others are held fixed. A multilinear function on a product of polytopes is, in each coordinate block, linear, so an extremum over the product is attained at a point extremal in every block, i.e. at a product of vertices. □

A **cell** is one level per level port. For each cell, the certificate records the interval $[lo_c, hi_c]$ that the instrument gives for each query over every belief consistent with the cell: by Proposition 10 this is the minimum and maximum over the cell's vertex worlds, each one exact instrument run. With no belief level at all, the vertices are the *pure* worlds and the bound is $\min_w$ and $\max_w$ of $\mu_w(Q)$ — the numbers the brute referee already enumerates. The sheet is finite: a `one` group of k members confined to a box contributes at most $k \cdot 2^{k-1}$ vertices, an independent atom two.

Corollary 11 (certification is preserved)**.** *The certificate of a program with belief ports is the program's own certificate (§9.4, keyed by* `program_key`*) together with the finite cell sheet, keyed additionally by the canonical line of the belief and level declarations. The port selects rows of a table that was certified without it; it never adds a cell.*

The honest boundary is where the hypothesis of Proposition 10 fails: a CHOICE weight that reads a belief port (the fetch rule's `@w look`) makes the trace measure itself depend on the belief as a *ratio* of port values. The function is then rational, not multilinear, and the vertex bound is not sound. Such a cell is listed *uncertified (choice reads a belief).* Both forms are legal; the certificate tells them apart, and the certified form gates the decision on a belief *level* with static weights.

8.7 The rewrites: on the instrument's net, never on the tick

The tick has no net; it samples one step. The instrument's net is static — the beliefs, the ground program, h and Q are fixed before it runs — so the proof-net rewrites of EFD [EFD26] apply to it, each keeping the exact rational and each checked against the brute referee: *weakening* (a group no ground event touches sums to one and is dropped), *deferred draws* (a group is drawn at the first state where its presence could matter, exact because the draw is independent of the trace so far), *absorption* (a state where Q holds stops), *merging* (equal states sum — the frontier is the cut-net's interface), and *the cone* (restriction to the outcome's influence cone when its closure holds). None of them is a semantic choice; each is Theorem 9's sum evaluated in a cheaper order.

8.8 Ports by step: the map's forecast as rows

A belief that moves along the horizon — a beast approaching, a car changing lane — is a hidden chain, and tracking it inside the instrument would be filtering, which §8.5 leaves in the map. The cheap form that pays today: the host computes its forecast *outside* the instrument, one row per step, and the instrument reads row t at step t. A rule factor prices the event chosen at step t by row t; a presence draw and a belief level read row 0; a row past the table's end repeats the last, so the frozen table is the one-row case and nothing earlier changes meaning.

Two points decide what a row may carry. A rule factor is a *per-step hazard* — the probability of the move at this step given where things are — never a marginal: a marginal ("nearer than now at step t") read as a factor at every step compounds. And a quantity constant over the horizon — an intent: hostile, wandering, fleeing — is better a categorical *belief* drawn once per world than a by-step port: the intent then persists along each trace (a hostile beast closes at every step, not at the mixture's average rate), Proposition 10 holds for it, and only the hazards conditioned on the intent move by step:

$$\mathit{close}: \quad \mathit{beast}(B,Q) \otimes \mathit{next}(P,Q) \otimes !\mathit{intent}(B,I) \multimap \mathit{beast}(B,P) \qquad \texttt{@w closing}(B,I).$$

On a lane with a beast believed at three cells, the risk of meeting it within eight events under a frozen hazard, a rising hazard by step, and the last row held frozen are 0.207, 0.294 and 0.346: the moving belief lies between its first and last rows, as it must.

With a by-step port, multilinearity is lost: the rows multiply along every trace and are functions of the forecast. The certificate then holds *over the initial levels* — the cells are the tick's levels, as before — and the forecast that turns the tick's belief into rows is part of the sealed bundle (§2.4), listed as *certified over the initial levels, predict in the bundle*, never by the vertex bound.

8.9 The aim: ports as functions, and the horizon

The form this section stops short of is a port bound as a *function* of the step and the state — the belief as a hidden Markov chain inside the instrument, updated by what each trace observes. The exact rational semantics and the cut carry over to it; Proposition 10 does not, and the certificate would have to be stated over the forecast's own parameters. The language refuses it today, by name, so that every program that compiles is one the cell sheet covers. It is open problem 5 (§13).

9 The compiled pipeline (systems)

`.llp` source (rulescript: Ceptre-core rules, typed header, ports,
includes with RDDL-style domain/instance lifting)
↓ parse → lossless AST (`parse(pretty(`a`))` $= a$)
↓ compile → catalog + ports; typecheck; admission
(packs: containment by construction — a pack may only *add*,
never touch base namespaces)
typed ground/unrolled **normal form** ← stages/quiescence lowered away (Prop. 6)
↓
MLL^{B}-shaped IR: occurrence net = CHOICE sites (categorical CPT boxes
over declared scopes) + SSA atom occurrences (binary wires)
↓ IR-level analyses: contextual typing; Bayesian proof-net check (Prop. 7);
canonical hash (`program_key`); COST CERTIFICATE (induced width + factor-ops,
symbolic — §9.4); junction tree (AOT)
↓ generated back-ends:
exact reachable-support filter (+ policy hook) [Thm 2]
factored elimination / junction tree (mid-scale exact)
fixed-N sampler (deterministic cost, same measure)
THTS planner (same measure, budgeted)
C++ IR interpreter (deployment; §9.6)
BN/factor-graph export (UAI + DOT; §9.7)

9.1 The knowledge-compilation frame

The nearest cousin is ProbLog → d-DNNF/SDD: compile once into a form whose evaluation is polynomial in the form's size, amortized over queries. MLL^{B} **is to LL-MSR what SDD circuits are to PLP** — with the difference that our target form is also a *published semantics* (EFD's nets), not only a data structure. To our knowledge no prior system compiles a staged stochastic multiset-rewriting language into proof-net-shaped factor graphs; EFD's own pipeline starts from BNs.

9.2 Evaluation: where the cost is, and what bounds it

Where does a program's evaluation spend its time? In two stages that obey two different bounds, and it pays to keep them apart because only one of them is covered by the proof-net theory.

Grounding is a join, before any net exists. A rule r has typed variables $x_1 \dots x_n$ and guards $g_1 \dots g_m$ (Horn premises). Its ground events are the substitutions for which every guard is derivable:

$$|\text{ground}(r)| \;=\; \left|\{\theta \in \textstyle\prod_i \tau_i : \forall j.\ \Sigma \vdash g_j\theta\}\right| \;\le\; \textstyle\prod_i |\tau_i|,$$

with equality exactly when no guard prunes. The bound is the program's, not the grounder's: a room of c cells whose adjacency is a predicate rather than a table emits $32\,c^2$ events for four rules (1.3 million at $c = 200$), and no grounder can emit fewer. Any grounder must at least read the facts its guards name and write its output, $\Omega(|\text{facts read}| + |\text{ground}(r)|)$. *Guard-first binding* reaches that bound for acyclic guard bodies: each variable's domain is filtered by every fact-table guard that names it as soon as it is bound, a semi-join per variable, so a prefix no row can complete is never extended. The alternative that answers a guard at its last variable is join-before-filter twice over — the type product before the guard, the whole fact list before the constants — and costs c^3 against an output of $8c$. A guard body with a cycle (a triangle $e(X,Y) \otimes e(Y,Z) \otimes e(Z,X)$) is bounded instead by the fractional edge cover

of its hypergraph (the AGM bound), and the compiler reports it so that the cost is never a surprise.

cells (static form)	200	450	840
guard at its last variable	22 s	248 s	29 min
guard-first binding	37 ms	94 ms	0.19 s

Emission and inference are EFD's object. Once grounded, the program's net exists, and from here the proof-net reading of [EFD26] applies verbatim: a site is a box, an occurrence edge is a cut, the elimination order is a sequence of cut expansions, and exact inference costs $O(n \cdot 2^W)$ in the induced width W of §9.4. The two stages do not trade against each other. Every room-shaped program, whatever its size, emits the same net at a given horizon (the agent reaches the same cells), with a width far past exact inference; a program written over *roles* — k places the cells are bound to — has width 8 at the same horizon. The width is chosen by the program, as the event count is; what the compiler chooses is only the order in which the program's own cost is paid.

Grounding cost is not inference cost, but the event count prices everything downstream once: the load and memory of every runtime (about a microsecond and a few hundred bytes per event), and the size of each CHOICE site's table in the net. A turn of the interpreter, by contrast, is bounded by the candidates weighed at that state, and an index by atom makes it independent of the program's size.

9.3 The factored emission and the `reads` contract

The emission (per micro-step): constants propagate — control atoms (chain and stage tokens) fold away, *branch-independence itself* being the criterion, with no control/state annotation anywhere. Steps whose candidates or scopes touch branch-dependent atoms become **sites**: categorical outcome variables whose CPT rows are the normalized CHOICE weights, with registry bindings evaluated on synthesized snapshots restricted to their **declared scope** — the language-level contract `weight f(C) reads {rb(C), prev(D) if adj(C,D)}`, guards enumerated against the Horn world. The snapshot object *raises* on any out-of-scope read: the contract has teeth, and Theorem 4 is why it must (an undeclared read breaks footprint stability). Writes become SSA versions via deterministic factors. Conservative static checks (multiplicities, schedule, double production) reject at admission; constructs static analysis cannot decide (stalled tiers, produce-onto-maybe-present) are emitted as zero-mass rows and indicator guards, and the evaluation-time total-mass check $Z = 1$ converts any *reachable* violation into an error. Conservative statics, exact runtime detection, never a silent approximation.

9.4 Certificates and the real-time doctrine

`certificate(query)` computes induced width and total factor-ops from IR scopes alone — no tables built, no binding called ([pinned]: a counting registry observes zero calls at admission). Budgets are platform-independent factor-ops, converted by one measured constant per machine; over-budget queries fall down the evaluator ladder (exact-support → factored → fixed-N sampling, whose cost is deterministic by construction). Measured tracking on Game of Life instances: certified ops 10^4 / 5×10^6 / 7×10^7 against measured 0.00 / 0.04 / 0.40 s. The junction tree makes the elimination structure itself AOT: built once per `program_key`, two-pass calibration per evidence; `unknowns` lift initial atoms to variables so *one* compiled structure serves every tick of a receding-horizon loop (the bounded-slot-template deployment story), with per-tick evidence as point factors.

9.5 Honesty: what the cliff is and is not

The exact filter dies where reachable support explodes — synchronous IPPC domains reach 2^n at $n \approx 14$–18. The factored back-end restores exactness where induced width stays local (ring topologies, bounded neighborhoods: $n = 24$ two-tick GoL in seconds), and *certifiably refuses* where it does not (the certificate is the refusal). The sampler and THTS carry the rest with fixed budgets.

Measured on one machine, in the Python referee:

evaluator	topology / instance	cost	note
exact support filter	sync ring, 1-tick proj., $n = 6$	< 0.01 s	support 2^6
	$n = 12$	0.65 s	2^{12}; clean $\sim 4\times$ per
	$n = 14$	3.64 s	2^{14} — per-step plann
	$n = 16$	18.3 s	2^{16} — per-step plann
factored / junction tree	sync ring, $n = 16$, two ticks (96 micro-steps)	< 60 s	exact past the filter's
	GoL ring, $n = 24$, two ticks	< 30 s	exact; support filter
	GoL instance 5 ($n = 16$), live play	65 ms/decision	certificate-priced, rea
fixed-N sampler	sync ring, $n = 14$	1.1 s (abs err 0.008)	vs 3.8 s exact — the
	sync ring, $n = 20$	2.1 s	exact infeasible at 2^2

The pattern the table quantifies: the exact filter's exponent is the physics of support enumeration (topology-independent — a ring at $n = 16$ costs the same 2^{16} as a clique); the factored evaluator's cost tracks *induced width*, so it is topology-dependent — local neighborhoods (rings, grids) stay exact for horizons and sizes the filter cannot touch, while densely-coupled domains blow the width and are *refused at admission* by the same certificate that admits the local ones (certified ops vs measured seconds calibrate at one constant, §9.4). The sampler's cost is deterministic by construction and carries what both exact evaluators refuse. On IPPC instances where exactness is in reach, the planner is provably optimal (exact expectimax over the true measure: Navigation, all tested instances; CrossingTraffic instance 1); Overcooked layouts reach expert parity (185/185); nothing is claimed against PROST-class planners on the large instances — that comparison is [open] and would be sampling-based.

9.6 C++ as IR interpreter; the compiler as verification toolchain

The deployment decision inverts the usual port: C++ never gains a grounder, typechecker or min-fill. The compiled artifact (structure + AOT schedules, serialized on the golden wire) is interpreted by a ∼300-line dependency-free table-ops loop; CPT rows are filled at run time by the registry bindings (they read the live world; that is the one thing that must be native). Admission happened on the authoring machine; the vehicle runs pre-certified artifacts with deterministic cost. The authoritative compiler is `metisc` (OCaml, standard library only, one static binary, from lexing to the reference sampler); the Python toolkit is the independent exact referee. Verification is a chain of custody: interpreter $\doteq$ filter (Thm 1–2 gates) $\doteq$ factored/junction tree (10^{-9} parity, whole-tick and mid-cycle) $\doteq$ exported artifact (independent brute-force reader) $\doteq$ C++ interpreter (golden parity on every corpus program, in both toolchains) — with pgmpy belief propagation as an external referee on the exported net.

9.7 Export: the EFD direction as a back-end

`to_uai` dumps the emitted net as a standard factor graph (UAI MARKOV; pgmpy/libDAI/merlin all read it), evidence baked in. This is the EFD direction — program → net → BN — ob-

tained *for free* because the IR already is the net; it doubles as external validation and as the data interface for parameter estimation (§10).

10 Learning: packs for structure, ports for parameters

One invariant: learning never edits a rule in place. The catalog stays the single reviewable, provenance-carrying artifact; every learned artifact passes the same admission gates as a hand-authored one.

Structure (surprise → synthesis). Zero predictive mass is the structural signal *by construction*: CHOICE weights multiply, so no parameter update can create missing support — a missing behavior is a missing rule, never a wrong number. Unmodeled observations produce a small deterministic record ({`program_key`, artifact versions, init multiset, observation, Z}); because evaluation is deterministic given these, the offline side re-derives full context (support, event posteriors, MAP explanation) — the vehicle logs pointers, not traces. Candidates (template enumeration over the typed vocabulary; the type system is the hypothesis-space bound) enter as `provenance=surprise` packs through containment admission.

Parameters (notching). By Theorem 4's factorization, each site is a multinomial over its declared scope — the emission literally exhibits the sufficient-statistic tables. Observed traces give per-site counts (conjugate Dirichlet updates; the hand prior and its strength are the pseudo-counts); partially observed traces give expected counts via exact smoothing (event posteriors under soft evidence) — EM whose E-step is free. Hypothesis-level selection over whole behavior rows is exact model filtering (posterior fed back as next prior; planted-row identifiability ≥ 0.9 recovery [`pinned`]). Learned values ship as overlay packs with a human-reviewable row sheet (`prior` → `posterior (`n`,` α_0`)` per scope assignment); the vehicle never learns in flight. The batch interface to external estimators is (UAI net, traces) — data, not an RL arena; the simulators are trace generators for it.

The loop. The loop that proposes packs — a language model drafting rules from surprise records, a gate admitting or refusing them — is the catalog paper's subject. Here two facts suffice: whatever the model proposes passes the same admission as a hand-written rule, and the world is driven by the admitted program, never by the model. Learning changes the program; it never retrains anything.

11 Mechanization

A Lean 4 development over mathlib ships in the public language repository, directory `formal/`. This paper contains no proof-assistant code: the development is the reference, and every statement below names its Lean identifier. The development is `sorry`-free, and the main theorems depend only on Lean's three standard axioms (`propext`, `Classical.choice`, `Quot.sound`), checked with `#print axioms`. The design principle mirrors the compiler's: formalize the *content* of a theorem at the level of abstraction where its mathematical difficulty lives, and leave the bookkeeping the implementation already pins (parsers, hashing, wire formats) to the golden gates.

Theorem 3, in full (`Formal/Spn.lean`). Ground programs are formalized as name-carrying lists of clauses and links over ambient types of atoms, stages and names; the SPN class as tuples whose transition components are multisets of content records; the denotation forgets

names and list order; structural congruence is the existence of a renaming injective on the program's names together with list permutations. The development proves: `denote_congr` (invariance: congruent programs denote equal SPNs), `denote_readBack` (surjectivity: an explicit fresh-naming section, $[\![U(N)]\!] = N$ on the nose), and `congr_of_denote_eq` (injectivity up to congruence, for well-formed programs — distinct clause and link names); together `denote_eq_iff_congr`, the bijection of Theorem 3. The delicate steps of the paper proof reappear precisely where expected: the duplicate-content matching is a permutation argument on lists with equal content multisets, and the empty-stage and multiset-not-set remarks are forced by the statement failing (as it should) under the alternative definitions.

Theorem 1, counting kernel (`Formal/Counting.lean`). An instance of clause r at marking m is formalized per species: an injection of consume slots into tokens and a function from read occurrences to tokens. `card_instance` proves the instance count equals $\varphi(m, r)$ — the falling factorial on consumed species (via $\#(\alpha \hookrightarrow \beta) = |\beta|^{\underline{|\alpha|}}$) times the power on read species — and `pushforward_weight` proves the per-clause pushforward identity $\sum_{\iota \in \mathrm{Inst}(r,m)} w_r = w_r \cdot \varphi(m, r) = W_r(m)$, i.e. the interpreter's instance-uniform draw and the DBN's CHOICE weights induce the same one-step clause distribution. This is the substantive combinatorial content of Theorem 1; the surrounding product induction is routine and remains on paper.

Theorem 2 (`Formal/Filter.lean`). Finitely-supported nonnegative state distributions with a finite-branching stochastic kernel; `filter_eq_marginal` proves by induction that the forward filter computed over the (finite) support at each step equals the t-step pushforward marginal — restriction to the reachable support loses no mass.

Theorem 4, descent skeleton (`Formal/Trace.lean`). Events with an independence relation, a deterministic update, and a local weight function; the two hypotheses of the footprint-stability lemma — independent events commute as state transformers, and each event's weight factor is unchanged by executing an independent event first — are taken as the definition of independence-compatibility. `weight_exec_swap` proves the run weight (product of local factors along the run) and final state are invariant under adjacent transposition of independent events, and `traceEquiv_weight_eq` / `traceEquiv_exec_eq` extend this to the Mazurkiewicz equivalence closure: the measure descends to trace classes. The concrete instantiation — that the factored emission's sites and declared scopes satisfy the two hypotheses (the paper's footprint-stability lemma) — is exactly the `reads`-contract argument of Theorem 4 and remains on paper, pinned operationally by the emission's rejection gates.

Theorem 9 and the cut bound (`Formal/Belief.lean`). Over the same finite state space and rational masses as `Filter.lean`: `marginal_sum` proves the forward filter linear in its initial distribution, and `hit_mixture` that the instrument started from the world mixture returns the mixture of the single-world answers for any kernel, outcome set and horizon — with the absorbing kernel `absorb`, the first-passage number of §8.5. `hit_cut_bounds` proves the interval of §8.5: a filter that sets states aside at each step returns $lo \le \text{exact} \le lo + CUT$ under a nonnegative stochastic kernel, through two invariants of the dropped part of the distribution — it stays nonnegative (`dropped_nonneg`) and its total is exactly the mass set aside (`dropped_total`). Proposition 10 (multilinearity) remains on paper.

What "automated" means. A belief table's side conditions — every group's masses nonnegative and summing to one, a level port's cut points strictly increasing in $(0, 1)$ — are a decidable predicate `Sheet.ok` on concrete data, closed by normalization (`norm_num`)

in the development's examples. That is the form the compiler can emit per package, next to the goldens: a statement per table, checked by the kernel, with Theorem 9 proved once above them. What cannot be emitted this way is a port as a function of the step (§8.8): the language refuses it, so there is nothing to check.

What is deliberately not mechanized. Propositions 6 and 7 are statements about the emission as implemented; their assurance story is the admission-time checker that runs at every serialization ([pinned]), which checks the actual artifact rather than a model of it. Mechanizing the checker itself (verify the verifier) and Conjecture 8's compositional typing are the natural next targets (§13).

12 Related work and provenance

The direct anchor. EFD [EFD24, EFD26]: BNs $\leftrightarrow$ $\mathrm{MLL}^{\mathrm{B}}$ proof-nets, inference = net evaluation in probabilistic coherence spaces, cost-matched; [DEF26] adds compositional typing (quantum and classical). Acclavio–Maieli [AM26]: proof *search* in probabilistic LL computing BN inference — the operational face. Our audit trail is explicit: the $\multimap$-as-V-joint observation predates our contact with this literature but was never published by us; priority on the static correspondence is theirs. What is ours is the dynamic, staged, executable direction (§1 items 1–9) and its compilation.

Foundations used. LL$\leftrightarrow$Petri: [GG89, EW93]; MSR/Ceptre lineage: [CS09, Mar15, CLF]; probabilistic coherence spaces and weighted relational models: [Gir04, DE11, EPT18, LMMP13]; Markov categories: [Fri20, CJ19]; true-concurrency probability: [AB06, Win13, CCPW18, Paq21]; GSPN/mass action: [Gil77, GSPN]; knowledge compilation: ProbLog $\rightarrow$ d-DNNF/SDD [FdB+15, Dar11]; classical-logic contrast: Markov Logic Networks [RD06] ground FOL into Markov nets — the linear analogue sat unoccupied until the EFD line; planning-as-inference: [TS06, Lev18]; mechanization: Lean 4 [MU21] and mathlib [mathlib20]. Behavior-tree engineering contrast: BT.cpp gives industry the component model but no measure; our framework keeps the component seams (registry, blackboard, ports) and adds semantics, and its interface packages are the contract half of that component model.

The belief side. A program with belief ports is a finite-horizon belief MDP [KLC98] evaluated at one tick over a finite quantisation. What is particular here is that the quantisation is the program's *input type* — the level atom — so the belief MDP is certified by a finite sheet over cells rather than by a bound over a continuous belief space, and the filter that moves the belief stays outside the certified object.

13 Open problems

1. The full trajectory-level statement of Theorem 1 (the product induction with normalization over enabled sets) and the concrete site/scope instantiation of Theorem 4 over a formalized emission.

2. Branching-cell measure beyond $\mathcal{F}$: import [AB06] to define trace-invariant weights for programs the emission rejects; does the general staged language admit a canonical net measure?

3. Conjecture 8: mechanize the [DEF26] well-behaved-composition typing derivations over the emitted artifact, and verify the verifier: re-prove the artifact checker of Proposition 7 in Lean against the formalized IR.

4. Proof theory of quiescence in the general class (subexponentials, fixpoints, or inhibitor-net semantics).

5. Ports as functions (§8): the instrument as a filter over a hidden chain, updated by what each trace observes. What replaces multilinearity, and over which parameters a certificate can then be stated.

6. The believed position of an unseen agent as a hazard: a predicted position counts as a hazard in a safety record and should not count as one in a verdict; a belief-aware form of the record.

7. Faithfulness: which queries determine a program in $\mathcal{F}$? And the engineering comparisons — sampling-based planners on large benchmark instances, complementary-pair merging and lifted emission — which this paper does not claim.

References

[AB06] S. Abbes, A. Benveniste. True-concurrency probabilistic models: branching cells and distributed probabilities. 2006/08.

[AM26] M. Acclavio, R. Maieli. Probabilistic Linear Logic Programming with an Application to Bayesian Network Computations. arXiv:2601.13270.

[CCPW18] S. Castellan, P. Clairambault, H. Paquet, G. Winskel. The concurrent game semantics of probabilistic PCF. LICS 2018.

[CJ19] K. Cho, B. Jacobs. Disintegration and Bayesian inversion via string diagrams. MSCS 2019.

[CLF] K. Watkins, I. Cervesato, F. Pfenning, D. Walker. A concurrent logical framework. CMU-CS-02-101.

[CS09] I. Cervesato, A. Scedrov. Relating state-based and process-based concurrency through linear logic. Inf. & Comp. 2009.

[Dar11] A. Darwiche. SDD: a new canonical representation of propositional knowledge bases. IJCAI 2011.

[DE11] V. Danos, T. Ehrhard. Probabilistic coherence spaces as a model of higher-order probabilistic computation. Inf. & Comp. 209(6), 2011.

[DEF26] R. Di Guardia, T. Ehrhard, C. Faggian. Quantum Bayesian Networks: Compositionality and Typing via Linear Logic. arXiv:2604.26059.

[EFD24] T. Ehrhard, C. Faggian, R. Di Guardia. Bayesian Networks and Proof-Nets: a Proof-Theoretical Account of Bayesian Inference. arXiv:2412.20540.

[EFD26] R. Di Guardia, T. Ehrhard, A. Evrard, C. Faggian. Bayesian Networks and Proof-Nets: the Proof-Theory of Bayesian Inference. arXiv:2602.04045.

[EPT18] T. Ehrhard, M. Pagani, C. Tasson. Full abstraction for probabilistic PCF. J. ACM 2018.

[EW93] U. Engberg, G. Winskel. Linear logic on Petri nets. 1990/93.

[FdB+15] D. Fierens, G. Van den Broeck, et al. Inference and learning in probabilistic logic programs using weighted boolean formulas. TPLP 2015.

[Fri20] T. Fritz. A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics. Adv. Math. 2020.

[GG89] C. Gunter, V. Gehlot. Nets as tensor theories. 1989.

[Gil77] D. Gillespie. Exact stochastic simulation of coupled chemical reactions. J. Phys. Chem. 1977.

[Gir87] J.-Y. Girard. Linear logic. Theoretical Computer Science 50(1), 1987.

[Gir04] J.-Y. Girard. Between logic and quantic: a tract. 2004.

[GSPN] M. Ajmone Marsan et al. Modelling with Generalized Stochastic Petri Nets. Wiley 1995.

[KLC98] L. P. Kaelbling, M. L. Littman, A. R. Cassandra. Planning and acting in partially observable stochastic domains. Artificial Intelligence 101, 1998.

[Kol33] A. N. Kolmogorov. Grundbegriffe der Wahrscheinlichkeitsrechnung. Springer 1933.

[Lev18] S. Levine. Reinforcement learning and control as probabilistic inference. arXiv:1805.00909.

[LMMP13] J. Laird, G. Manzonetto, G. McCusker, M. Pagani. Weighted relational models of typed lambda-calculi. LICS 2013.

[Mar15] C. Martens. Ceptre: a language for modeling generative interactive systems. AIIDE 2015.

[mathlib20] The mathlib Community. The Lean mathematical library. CPP 2020.

[MU21] L. de Moura, S. Ullrich. The Lean 4 theorem prover and programming language. CADE 2021.

[Paq21] H. Paquet. Bayesian strategies: probabilistic programs as generalised graphical models. ESOP 2021.

[RD06] M. Richardson, P. Domingos. Markov logic networks. MLJ 2006.

[TS06] M. Toussaint, A. Storkey. Probabilistic inference for solving discrete and continuous state Markov decision processes. ICML 2006.

[Win13] G. Winskel. Distributed probabilistic and quantum strategies. MFPS 2013.